ATIVIDADES DA UNIDADE CURRICULAR

Unidade Curricular: Programação para Internet I

Semestre 3 – 60h

Sumário

1 Visão Geral da Unidade Curricular **2**

2 Roteiro Prático 01 — Fundamentos da Web, HTML5 e Versionamento **4**

2.1 Introdução 4

2.2 Parte 1 — Fundamentos da Internet 5

2.2.1 Como funciona uma página web? 5

2.3 Parte 2 — Estrutura Base do Documento HTML5 6

2.3.1 Estrutura mínima obrigatória 6

2.4 Parte 3 — Comandos Básicos HTML5 7

2.5 Parte 4 — Trabalhando com Links 9

2.5.1 Exemplo 9

2.6 Parte 5 — Versionamento com Git e GitHub 10

2.7 Checklist Final de Verificação 11

3 Roteiro Prático 02 — HTML5 Semântico, CSS Externo e Estrutura Profissional **12**

3.1 Introdução 12

3.2 Fundamentação Teórica — HTML5 Semântico 12

3.3 Principais Tags Estruturais do HTML5 13

3.4 Boas Práticas 13

3.5 Parte 1 — Organização Profissional do Projeto 15

3.6 Parte 2 — Estrutura HTML Semântica 15

3.7 Parte 3 — Introdução ao CSS Externo 17

3.8 Parte 4 — Inserindo Imagens 19

3.9 Checklist Final 19

3.10 Fundamentação Complementar — Propriedades CSS Essenciais 21

3.11 Principais Propriedades CSS 21

3.12 Dicas Essenciais — Git e GitHub 22

3 Roteiro Prático 02 — HTML5 Semântico, CSS Externo e Estrutura Profissional

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Utilizar corretamente HTML5 semântico;
- Organizar múltiplas páginas interligadas;
- Criar e conectar um arquivo CSS externo;
- Trabalhar com imagens utilizando caminhos relativos;
- Estruturar diretórios de forma profissional;
- Aplicar versionamento incremental com Git.

Introdução

No roteiro anterior construímos uma página HTML básica e iniciamos o versionamento com Git.

Agora avançaremos para um modelo mais próximo de aplicações reais, organizando um mini-site com múltiplas páginas, estrutura semântica adequada e separação entre conteúdo (HTML) e estilo (CSS).

A separação entre estrutura e apresentação é um dos pilares do desenvolvimento web moderno. O HTML é responsável pela organização do conteúdo, enquanto o CSS define a aparência visual da aplicação.

Neste roteiro construiremos um mini-site institucional com organização profissional.

Fundamentação Teórica — HTML5 Semântico

O HTML5 introduziu uma abordagem mais semântica para organização de páginas web. Isso significa que as tags passaram a descrever o significado estrutural do conteúdo.

A semântica melhora:

- Organização do código;
- Acessibilidade;
- SEO (Search Engine Optimization);
- Manutenção e escalabilidade do projeto.

Principais Tags Estruturais do HTML5

Tag	Função Estrutural
<header>	Cabeçalho da página ou seção.
<nav>	Área de navegação principal.
<main>	Conteúdo principal da página.
<section>	Agrupamento temático de conteúdo.
<article>	Conteúdo independente (post, card, notícia).
<aside>	Conteúdo complementar ou lateral.
<footer>	Rodapé da página ou seção.
<link>	Conexão com arquivos externos (CSS).
<img>	Inserção de imagem (com atributo alt obrigatório).

Boas Práticas

Evite estruturar páginas apenas com . Utilize as tags semânticas para tornar o código mais claro e profissional.

Dicas Úteis — VS Code

Atalhos Importantes no VS Code para HTML

- Digite `!` e pressione `TAB` para gerar automaticamente a estrutura básica do HTML5.
- Use `ALT + SHIFT + F` para formatar automaticamente o código.
- Utilize `CTRL + /` para comentar ou descomentar linhas.
- Use `CTRL + Espaço` para ativar sugestões automáticas.
- Utilize `CTRL + S` frequentemente para salvar alterações.

Dica Profissional: Acostume-se a trabalhar com código bem indentado e organizado desde o início. Isso faz parte da avaliação formativa.

Dicas Úteis — VS Code

Sequência Recomendada de Uso do Git

Sempre que finalizar uma etapa do trabalho, utilize a seguinte sequência:

```
git status
git add .
git commit -m "Descrição clara da alteração"
git push
```

Boa prática:

- Faça commits pequenos e frequentes;
- Utilize mensagens descritivas;
- Evite acumular muitas alterações em um único commit.

Parte 1 — Organização Profissional do Projeto

Estrutura recomendada:

```
prog-internet-2026/

 roteiro02/
     index.html
     sobre.html
     contato.html
     assets/
         css/
            style.css
         img/
             perfil.jpg
```

Atividade 1 — Criando Estrutura Base

- Criar pasta `roteiro02`;
- Criar os três arquivos HTML;
- Criar pasta `assets/css`;
- Criar pasta `assets/img`;
- Criar arquivo `style.css`.

Micro-Commit 1

```
git add .
git commit -m "Criando estrutura profissional roteiro 02"
```

Parte 2 — Estrutura HTML Semântica

No arquivo `index.html`:

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Mini-Site</title>
    <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>

<header>
    <h1>Meu Mini-Site</h1>
</header>

<nav>
    <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="sobre.html">Sobre</a></li>
        <li><a href="contato.html">Contato</a></li>
    </ul>
</nav>

<main>
    <section>
        <h2>Conteúdo Principal</h2>
        <p>Texto da seção.</p>
    </section>
</main>

<footer>
    <p>Seu nome - 2026</p>
</footer>

</body>
</html>
```

Atividade 2 — Implementando Estrutura

- Implementar estrutura nas três páginas;
- Garantir navegação funcional;
- Testar todos os links.

Micro-Commit 2

```
git add .
git commit -m "Implementando estrutura semantica completa"
```

Parte 3 — Introdução ao CSS Externo

No arquivo `style.css`:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
}

header {
    background-color: #003366;
    color: white;
    padding: 15px;
}

nav ul {
    list-style: none;
}

nav li {
    display: inline;
    margin-right: 15px;
}

footer {
    margin-top: 20px;
    font-size: 14px;
}
```

Atividade 3 — Aplicando CSS

- Criar e conectar arquivo CSS;
- Verificar funcionamento nas três páginas;
- Ajustar estilos conforme necessário.

Micro-Commit 3

```
git add .
git commit -m "Adicionando estilo CSS externo"
```

Parte 4 — Inserindo Imagens

No arquivo `index.html`:

```
<img src="assets/img/perfil.jpg" alt="Foto de perfil">
```

Atividade 4 — Trabalhando com Imagem

- Inserir imagem na página inicial;
- Utilizar atributo alt;
- Garantir caminho relativo correto.

Micro-Commit 4

```
git add .
git commit -m "Adicionando imagem ao mini-site"
```

Checklist Final

Verificação Estrutural

- Estrutura de diretórios organizada;
- HTML semântico implementado corretamente;
- CSS externo funcionando;
- Navegação funcional;
- Imagem carregando corretamente;
- Pelo menos 4 commits realizados.

Publicação Final no GitHub

Após revisar o checklist:

- Verifique se todos os arquivos estão salvos;
- Execute a sequência completa de versionamento;
- Certifique-se de que o repositório remoto está atualizado.

```
git status
git add .
git commit -m "Finalizacao roteiro 02"
git push
```

Resultado esperado:
Todos os arquivos do roteiro02 devem estar visíveis no GitHub.

Fundamentação Complementar — Propriedades CSS Essenciais

O CSS (Cascading Style Sheets) é responsável pela apresentação visual da página. Ele permite definir cores, espaçamentos, fontes, posicionamento e organização visual.

Principais Propriedades CSS

Propriedade	Função
color	Define a cor do texto.
background-color	Define a cor de fundo do elemento.
font-family	Define a família da fonte.
font-size	Define o tamanho da fonte.
margin	Define o espaçamento externo do elemento.
padding	Define o espaçamento interno do elemento.
border	Define bordas ao redor do elemento.
width / height	Define largura e altura do elemento.
display	Define o tipo de exibição (block, inline, inline-block).
text-align	Define alinhamento do texto.

Exemplo Prático

```
section {
    background-color: #f4f4f4;
    padding: 20px;
    margin: 10px;
}
```

Dicas Essenciais — Git e GitHub

O Git é a ferramenta de controle de versão utilizada nesta disciplina. Abaixo estão os principais comandos que você utilizará com frequência.

Comando	Função
git status	Mostra o estado atual do repositório (arquivos modificados, adicionados ou pendentes de commit).
git add .	Adiciona todos os arquivos modificados para a área de preparação (staging area).
git commit -m "mensagem"	Cria um novo commit com uma descrição clara da alteração realizada.
git push	Envia os commits locais para o repositório remoto no GitHub.
git pull	Atualiza seu repositório local com alterações feitas no repositório remoto.

6. FAQ – Problemas Comuns com Git e GitHub

Durante as atividades práticas com Git e GitHub, alguns erros podem ocorrer, principalmente em ambientes de laboratório Linux. Abaixo estão os problemas mais comuns e como resolvê-los rapidamente.

6.1 Erro: Please tell me who you are

Mensagem típica:

```
*** Please tell me who you are.
Run
  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"
```

Causa: O Git precisa identificar quem está realizando o commit. Nome e e-mail ainda não foram configurados na máquina.

Solução:

Execute no terminal:

```
git config --global user.name "Seu Nome Completo"
git config --global user.email "seu_email@exemplo.com"
```

Exemplo:

```
git config --global user.name "Maria da Silva"
git config --global user.email "maria@email.com"
```

Após isso, tente novamente:

```
git commit -m "Mensagem do commit"
```

Observação: Se o computador for compartilhado, pode-se configurar apenas para o repositório atual:

```
git config user.name "Seu Nome"
git config user.email "seu_email@exemplo.com"
```

—

6.2 Erro: "fatal: unable to auto-detect email address

Causa: Mesmo problema do item anterior: o Git não tem nome e e-mail configurados.

Solução: Configurar o usuário conforme indicado na seção 6.1.

—

6.3 Erro ao dar push – senha não aceita

Mensagem típica:

```
remote: Support for password authentication was removed
fatal: Authentication failed
```

Causa: O GitHub não permite mais autenticação usando apenas senha.
Soluções possíveis:
Opção 1 – Utilizar Token de Acesso (PAT):

- Gerar um token no GitHub.
- Usar o token no lugar da senha ao executar o `git push`.

Opção 2 – Configurar autenticação via SSH:

- Gerar chave SSH com:

  ```
  ssh-keygen -t ed25519 -C "seu_email@exemplo.com"
  ```

- Adicionar a chave pública nas configurações do GitHub.

—

6.4 Verificar configurações atuais do Git

Para conferir se o nome e e-mail estão configurados:

```
git config --list
```

Ou apenas as configurações globais:

```
git config --global --list
```

—

6.5 Erro: "nothing to commit

Mensagem típica:

```
nothing to commit, working tree clean
```

Causa: Nenhuma alteração foi adicionada à área de stage.

Solução:

Verificar o status:

```
git status
```

Adicionar arquivos:

```
git add nome_do_arquivo
```

Ou todos os arquivos:

```
git add .
```

Depois realizar o commit novamente.

—

6.6 Dica Importante

Sempre que ocorrer um erro:

- Leia atentamente a mensagem exibida no terminal.
- Utilize o comando `git status` para entender a situação atual do repositório.
- Evite repetir comandos sem compreender o erro.

Grande parte dos problemas com Git está relacionada à configuração inicial ou à autenticação.